# MRC Lecture 1 — Derivation Guide 2

# Matrix Multiplication: Computing All Pairwise Compatibilities

Sai T. Reddy — ETH Zurich / BIIE

## 1. Motivation

In Derivation Guide 1, we showed that the dot product computes the compatibility between one agent and one target. In molecular recognition, however, we rarely care about a single pair. An antibody repertoire contains millions of distinct antibodies, each potentially binding thousands of antigens. A transcription factor scans thousands of candidate binding sites across the genome. A transformer query attends to every key in the sequence simultaneously.

We need to compute **all pairwise compatibility scores at once**. Matrix multiplication does exactly this — it performs all pairwise dot products in a single operation. This is the mathematical basis for the computational efficiency of both transformer attention and Specificity Foundation Models.

## 2. Definition of Matrix Multiplication

**Definition.** Given a matrix $A \in \mathbb{R}^{m \times p}$ (with $m$ rows and $p$ columns) and a matrix $B \in \mathbb{R}^{p \times n}$ (with $p$ rows and $n$ columns), their product $C = AB$ is a matrix $C \in \mathbb{R}^{m \times n}$ where each entry is:

$$C_{ij} = \sum_{k=1}^{p} A_{ik} B_{kj}$$

In words: the entry in row $i$, column $j$ of the product is the **dot product of row $i$ of $A$ with column $j$ of $B$**.

**Dimension requirement.** The number of columns of $A$ must equal the number of rows of $B$. The shared dimension $p$ is consumed in the summation; the output has dimensions $m \times n$.

$$\underset{\text{¿}}{A} \times \underset{\text{¿}}{B} = \underset{\text{¿}}{C}$$

## 3. Worked Example 1: Full Computation

**Problem.** Compute $AB$ for:

$$A = \begin{pmatrix} 2 & 1 \\ 0 & 3 \\ 1 & 2 \end{pmatrix}, B = \begin{pmatrix} 1 & 4 & 0 \\ 3 & 1 & 2 \end{pmatrix}$$

$A$ is $3 \times 2$ and $B$ is $2 \times 3$, so the product $C = AB$ is $3 \times 3$.

**Step 1.** Compute row 1 of $C$ (row 1 of $A$ dotted with each column of $B$):

$$C_{11} = (2)(1) + (1)(3) = 2 + 3 = 5$$
$$C_{12} = (2)(4) + (1)(1) = 8 + 1 = 9$$
$$C_{13} = (2)(0) + (1)(2) = 0 + 2 = 2$$

**Step 2.** Compute row 2 of $C$ (row 2 of $A$ dotted with each column of $B$):

$$C_{21} = (0)(1) + (3)(3) = 0 + 9 = 9$$
$$C_{22} = (0)(4) + (3)(1) = 0 + 3 = 3$$
$$C_{23} = (0)(0) + (3)(2) = 0 + 6 = 6$$

**Step 3.** Compute row 3 of $C$ (row 3 of $A$ dotted with each column of $B$):

$$C_{31} = (1)(1) + (2)(3) = 1 + 6 = 7$$
$$C_{32} = (1)(4) + (2)(1) = 4 + 2 = 6$$

$$C_{33} = (1)(0) + (2)(2) = 0 + 4 = 4$$

**Step 4.** Assemble the result:

$$C = AB = \begin{pmatrix} 5 & 9 & 2 \\ 9 & 3 & 6 \\ 7 & 6 & 4 \end{pmatrix}$$

Each entry $C_{ij}$ is a dot product — row $i$ of $A$ with column $j$ of $B$. Nine dot products, computed in one matrix multiplication.

## 4. The Transpose Operation

Before applying matrix multiplication to molecular recognition, we need the **transpose**.

**Definition.** The transpose of a matrix $B \in \mathbb{R}^{n \times d}$ is the matrix $B^T \in \mathbb{R}^{d \times n}$ obtained by swapping rows and columns:

$$(B^T)_{ij} = B_{ji}$$

Row $i$ of $B$ becomes column $i$ of $B^T$.

**Example:**

$$B = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{pmatrix} \Longrightarrow B^T = \begin{pmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{pmatrix}$$

**Why the transpose matters.** In most data representations, each row is a data point and each column is a feature. If $Q$ has $m$ agents as rows and $K$ has $n$ targets as rows, both with $d$ features per row, then $Q$ is $m \times d$ and $K$ is $n \times d$. We cannot directly multiply $Q \times K$ because the inner dimensions do not match ($d \neq n$ in general). But $K^T$ is $d \times n$, so $Q K^T$ is well-defined and has dimensions $m \times n$. The transpose converts "targets as rows" to "targets as columns," enabling the pairwise dot products.

## 5. The Pairwise Similarity Matrix: $S = Q K^T$

**Setup.** Let $Q \in \mathbb{R}^{m \times d}$ be a matrix of agent embeddings, where row $i$ is the $d$-dimensional embedding vector $q_i$ for agent $i$. Let $K \in \mathbb{R}^{n \times d}$ be a matrix of target embeddings, where row $j$ is the $d$-dimensional embedding vector $k_j$ for target $j$.

**Claim.** The matrix product $S = Q K^T$ is the $m \times n$ matrix of all pairwise dot products:

$$S_{ij} = q_i \cdot k_j$$

**Proof.** By the definition of matrix multiplication:

$$S_{ij} = (Q K^T)_{ij} = \sum_{l=1}^{d} Q_{il} (K^T)_{lj} = \sum_{l=1}^{d} Q_{il} K_{jl}$$

Since $Q_{il}$ is the $l$-th component of agent $i$'s embedding and $K_{jl}$ is the $l$-th component of target $j$'s embedding:

$$S_{ij} = \sum_{l=1}^{d} (q_i)_l (k_j)_l = q_i \cdot k_j \quad \blacksquare$$

Entry $(i, j)$ of the similarity matrix is the dot product — the compatibility score — between agent $i$ and target $j$. One matrix multiplication computes all $m \times n$ pairwise scores simultaneously.

## 6. Worked Example 2: Molecular Similarity Matrix

**Problem.** Three antibodies have 4-dimensional embedding vectors:

$$Q = \begin{pmatrix} 1 & 0 & 2 & 1 \\ 0 & 1 & 1 & 2 \\ 2 & 1 & 0 & 1 \end{pmatrix}$$

Five antigens have 4-dimensional embedding vectors:

$$K = \begin{pmatrix} 2 & 1 & 1 & 0 \\ 0 & 2 & 1 & 1 \\ 1 & 0 & 3 & 1 \\ 1 & 1 & 0 & 2 \\ 0 & 1 & 2 & 0 \end{pmatrix}$$

Compute the full similarity matrix $S = Q\,K^T$.

**Step 1.** Confirm dimensions. $Q$ is $3 \times 4$, $K$ is $5 \times 4$, so $K^T$ is $4 \times 5$. The product $S = Q\,K^T$ is $3 \times 5$: three antibodies scored against five antigens.

**Step 2.** Compute $K^T$:

$$K^T = \begin{pmatrix} 2 & 0 & 1 & 1 & 0 \\ 1 & 2 & 0 & 1 & 1 \\ 1 & 1 & 3 & 0 & 2 \\ 0 & 1 & 1 & 2 & 0 \end{pmatrix}$$

**Step 3.** Compute row 1 of $S$ (Antibody 1 against all antigens):

$$S_{11} = (1)(2) + (0)(1) + (2)(1) + (1)(0) = 2 + 0 + 2 + 0 = 4$$
$$S_{12} = (1)(0) + (0)(2) + (2)(1) + (1)(1) = 0 + 0 + 2 + 1 = 3$$
$$S_{13} = (1)(1) + (0)(0) + (2)(3) + (1)(1) = 1 + 0 + 6 + 1 = 8$$
$$S_{14} = (1)(1) + (0)(1) + (2)(0) + (1)(2) = 1 + 0 + 0 + 2 = 3$$
$$S_{15} = (1)(0) + (0)(1) + (2)(2) + (1)(0) = 0 + 0 + 4 + 0 = 4$$

**Step 4.** Compute row 2 of $S$ (Antibody 2 against all antigens):

$$S_{21} = (0)(2) + (1)(1) + (1)(1) + (2)(0) = 0 + 1 + 1 + 0 = 2$$
$$S_{22} = (0)(0) + (1)(2) + (1)(1) + (2)(1) = 0 + 2 + 1 + 2 = 5$$
$$S_{23} = (0)(1) + (1)(0) + (1)(3) + (2)(1) = 0 + 0 + 3 + 2 = 5$$
$$S_{24} = (0)(1) + (1)(1) + (1)(0) + (2)(2) = 0 + 1 + 0 + 4 = 5$$
$$S_{25} = (0)(0) + (1)(1) + (1)(2) + (2)(0) = 0 + 1 + 2 + 0 = 3$$

**Step 5.** Compute row 3 of $S$ (Antibody 3 against all antigens):

$$S_{31} = (2)(2) + (1)(1) + (0)(1) + (1)(0) = 4 + 1 + 0 + 0 = 5$$
$$S_{32} = (2)(0) + (1)(2) + (0)(1) + (1)(1) = 0 + 2 + 0 + 1 = 3$$
$$S_{33} = (2)(1) + (1)(0) + (0)(3) + (1)(1) = 2 + 0 + 0 + 1 = 3$$
$$S_{34} = (2)(1) + (1)(1) + (0)(0) + (1)(2) = 2 + 1 + 0 + 2 = 5$$
$$S_{35} = (2)(0) + (1)(1) + (0)(2) + (1)(0) = 0 + 1 + 0 + 0 = 1$$

**Step 6.** Assemble the result:

$$S = Q\,K^T = \begin{pmatrix} 4 & 3 & 8 & 3 & 4 \\ 2 & 5 & 5 & 5 & 3 \\ 5 & 3 & 3 & 5 & 1 \end{pmatrix}$$

**Step 7.** Interpret:

| | Ag 1 | Ag 2 | Ag 3 | Ag 4 | Ag 5 |
|---|---|---|---|---|---|
| **Ab 1** | 4 | 3 | **8** | 3 | 4 |
| **Ab 2** | 2 | 5 | 5 | **5** | 3 |

| | Ag 1 | Ag 2 | Ag 3 | Ag 4 | Ag 5 |
|---|---|---|---|---|---|
| **Ab 3** | **5** | 3 | 3 | **5** | 1 |

Antibody 1 has the highest compatibility with Antigen 3 (score = 8). Antibody 2 is equally compatible with Antigens 2, 3, and 4 (score = 5 each). Antibody 3 prefers Antigens 1 and 4 (score = 5 each).
In a Specificity Foundation Model, each row of this matrix would be passed through the softmax function (the convergence equation) to convert raw scores into binding probabilities. We will derive this in Lectures 6 and 7.

## 7. Connection to Transformer Attention

The similarity matrix $S = Q K^T$ is the core computation in transformer attention. In the notation of Vaswani et al. (2017):

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{Q K^T}{\sqrt{d_k}}\right) V$$

This equation has three stages:
**Stage 1: Compute all pairwise scores.** $S = Q K^T$ — the matrix multiplication we derived above. This produces the $m \times n$ matrix of all agent-target compatibility scores. This is one operation.
**Stage 2: Normalize to probabilities.** $\text{softmax}\left(S / \sqrt{d_k}\right)$ — divide by the temperature $\tau = \sqrt{d_k}$ and apply softmax row-wise. Each row becomes a probability distribution over targets. This is the convergence equation applied to each agent independently. We cover softmax in Derivation Guides 4 and 5.
**Stage 3: Aggregate information.** Multiply the attention weights by the value matrix $V$ — each agent's output is a weighted average of target value vectors, where the weights are the binding probabilities. This step transfers information from targets to agents proportional to compatibility.
For an SFM, Stages 1 and 2 are the binding prediction: compute all pairwise binding energies (matrix multiplication) and convert them to binding probabilities (softmax). Stage 3 is used in the cross-attentive decoder for sequence generation (Lecture 7).

## 8. Computational Complexity

**Single dot product:** Computing $q_i \cdot k_j$ for $d$-dimensional vectors requires $d$ multiplications and $d - 1$ additions, which is $O(d)$ operations.
**Full similarity matrix:** Computing $S = Q K^T$ for $m$ agents and $n$ targets requires $m \times n$ dot products, each of cost $O(d)$. The total cost is $O(mnd)$.
**Why this matters for SFMs.** Consider ranking one antibody against $n = 10,000$ candidate antigens using $d = 512$ dimensional embeddings. The cost is $1 \times 10,000 \times 512 \approx 5 \times 10^6$ floating point operations — achievable in milliseconds on a modern GPU. By contrast, evaluating the same antibody against 10,000 antigens using a structural co-folding model (e.g., AlphaFold3) requires 10,000 independent co-folding runs, each taking minutes to hours. The difference in computational cost is orders of magnitude, and it traces directly to the fact that the SFM's compatibility score is a dot product computable via matrix multiplication.
**GPU parallelism.** Matrix multiplication is one of the most highly optimized operations in modern computing. GPU hardware (NVIDIA A100, H100) is specifically designed for large matrix multiplications, achieving peak throughput of hundreds of teraflops (trillions of floating-point operations per second). The entire SFM retrieval pipeline — encoding sequences, computing the similarity matrix, applying softmax — maps naturally to GPU-accelerated matrix operations.

## 9. Properties of Matrix Multiplication

Several properties of matrix multiplication are relevant for later lectures.

**Non-commutativity.** In general, $AB \neq BA$. The product may not even be defined in both orders (if the dimensions don't match), and even when both products exist, they generally differ. This means the direction of the computation matters: $Q K^T$ (agents as queries, targets as keys) computes agent-to-target scores; $K Q^T$ computes target-to-agent scores. In the SFM symmetric loss (Lecture 7), both directions are computed and averaged, precisely because binding free energy is symmetric ($\Delta G_{AB} = \Delta G_{BA}$) but the matrix multiplication is not.

**Associativity.** $(AB)C = A(BC)$. This allows choosing the most efficient order of multiplication. In the attention equation, $\text{softmax}\left(Q K^T / \sqrt{d_k}\right) V$ can be computed by first forming the $m \times n$ attention matrix and then multiplying by $V$, or (in certain approximations) by first computing $K^T V$ and then multiplying by $Q$.

**Transpose of a product.** $(AB)^T = B^T A^T$. The order reverses. This identity is used when deriving the symmetric loss (Lecture 7).

## 10. Worked Example 3: Reading a Similarity Matrix

**Problem.** Given the similarity matrix from Example 2:

$$S = \begin{pmatrix} 4 & 3 & 8 & 3 & 4 \\ 2 & 5 & 5 & 5 & 3 \\ 5 & 3 & 3 & 5 & 1 \end{pmatrix}$$

(a) Which antigen does Antibody 1 most likely bind?
(b) Which antibody does Antigen 4 most likely attract?
(c) Which antibody-antigen pair has the weakest interaction?

**Answers:**

(a) Read row 1: scores are $(4, 3, 8, 3, 4)$. The maximum is $S_{13} = 8$. Antibody 1 most likely binds **Antigen 3**.
(b) Read column 4: scores are $(3, 5, 5)$. Antibodies 2 and 3 are tied at $S_{24} = S_{34} = 5$. **Antibodies 2 and 3** are equally likely to bind Antigen 4.
(c) The minimum entry in the matrix is $S_{35} = 1$. The weakest interaction is between **Antibody 3 and Antigen 5**.

**Note:** These are raw dot-product scores, not probabilities. To obtain binding probabilities, each row would be passed through the softmax function independently. After softmax, Antibody 1's probability of binding Antigen 3 would be much higher than its probability of binding any other antigen, because softmax exponentiates the scores and amplifies differences (Derivation Guide 4).

## 11. Summary

| Concept | Definition | Role in MRC |
|---|---|---|
| Matrix multiplication | $C_{ij} = \sum_k A_{ik} B_{kj}$ | Performs all pairwise dot products in one operation |
| Transpose | $(B^T)_{ij} = B_{ji}$ | Converts "targets as rows" to "targets as columns" |
| Similarity matrix | $S = Q K^T$ | All pairwise agent-target compatibility scores |
| Entry $S_{ij}$ | $q_i \cdot k_j$ | Compatibility of agent $i$ with target $j$ |
| Complexity | $O(mnd)$ | Milliseconds on GPU for $m$ agents, $n$ targets |
| Non-commutativity | $Q K^T \neq K Q^T$ | Motivates the symmetric loss (Lecture 7) |

## 12. Checkpoint Questions

**Q1.** If $Q$ has shape $100 \times 512$ (100 antibodies, 512-dimensional embeddings) and $K$ has shape $5000 \times 512$ (5000 antigens, 512-dimensional embeddings), what is the shape of $S = Q K^T$? How many dot products does this represent?
*Answer:* $K^T$ is $512 \times 5000$, so $S = Q K^T$ is $100 \times 5000$. This represents $100 \times 5000 = 500,000$ pairwise dot products — every antibody scored against every antigen — computed in a single matrix multiplication.
**Q2.** In the similarity matrix $S$, what does reading across a single row tell you? What does reading down a single column tell you?
*Answer:* Reading across row $i$ gives the compatibility scores of agent $i$ with every target — this is the agent's "preference profile." Applying softmax to this row gives the agent's binding probability distribution. Reading down column $j$ gives the compatibility scores of every agent with target $j$ — this is the target's "attractiveness profile." Applying softmax to this column gives the probability that each agent selects this target. The SFM symmetric loss (Lecture 7) uses both directions.
**Q3.** Why does the SFM use $Q K^T$ (a matrix multiplication) rather than computing each dot product $q_i \cdot k_j$ in a loop?
*Answer:* Mathematically, both approaches give the same result. Computationally, matrix multiplication maps to highly optimized GPU kernels that exploit hardware parallelism. A loop computes dot products sequentially; a matrix multiplication computes them all in parallel. For large-scale molecular recognition (millions of agents, millions of targets), the difference in wall-clock time can be several orders of magnitude.
**Q4.** The attention equation divides $Q K^T$ by $\sqrt{d_k}$ before applying softmax. Based on what you know about the dot product, why might large $d_k$ cause problems?
*Answer:* From Derivation Guide 1, the dot product of two random $d_k$-dimensional vectors scales with $d_k$: the expected value of $q \cdot k$ grows as the number of dimensions increases (each term in the sum contributes positively on average). If the dot products become very large, the softmax function saturates — the largest score gets probability $\approx 1$ and all others get probability $\approx 0$. This is equivalent to very low temperature (Derivation Guide 5). Dividing by $\sqrt{d_k}$ rescales the scores to have variance $O(1)$ regardless of $d_k$, keeping the softmax in its sensitive regime. We derive this scaling in Lecture 2.

## 13. Connection to Future Lectures

- **Lecture 2:** The full transformer attention equation $\text{softmax}\left(Q K^T / \sqrt{d_k}\right) V$ is derived on the whiteboard. Matrix multiplication is Stage 1 of the three-stage attention pipeline.
- **Lecture 3:** In contrastive learning (InfoNCE), the similarity matrix $S$ between all pairs in a mini-batch is computed via matrix multiplication. The contrastive loss is defined over the rows and columns of this matrix.
- **Lecture 7:** The SFM architecture prescribes dual encoders producing $Q$ (agent embeddings) and $K$ (target embeddings). The matrix product $Q K^T$ computes all pairwise binding energies. The symmetric loss requires computing softmax over both rows and columns — this is where the non-commutativity of matrix multiplication connects to the symmetry of binding free energy.
- **Lecture 11:** The computational economics argument rests on the fact that an SFM computes all pairwise scores via a single matrix multiplication (seconds on a GPU), while structure-first methods require independent evaluations per pair (hours to days). The $O(mnd)$ scaling of matrix multiplication is the mathematical basis for this efficiency advantage.